

Keys, Relationships, and Joins

A join is a claim about the data.
Today you learn how to prove the claim.



One-to-many relationships can grow a result.

AI-Free Zone: write, inspect, and verify every query yourself.

Yesterday's clues need relationships

Day 1: four suspects

We could filter and summarize evidence one table at a time.

Today: connect evidence

Which person owns which vehicle? Which person attended which event?

Before belief: validate

What should one row mean?
What count should change?

What two tables would you connect first?

Also remember the call-center AgentID = -1: an orphaned key can quietly remove a row in an inner join.

Keys give a join its meaning

STATIONS

One row per station
Primary key: station_id

one station ← many trips

TRIPS

One row per bike trip
Foreign key: start_station_id

A key is not merely a column name. It tells you what can match and how often.

Choose the rows you need to preserve

INNER JOIN

Keeps rows with a match on both sides

Use it when only matched records answer the question.

LEFT JOIN

Keeps every row from the table on the left

Missing lookup details become NULL.

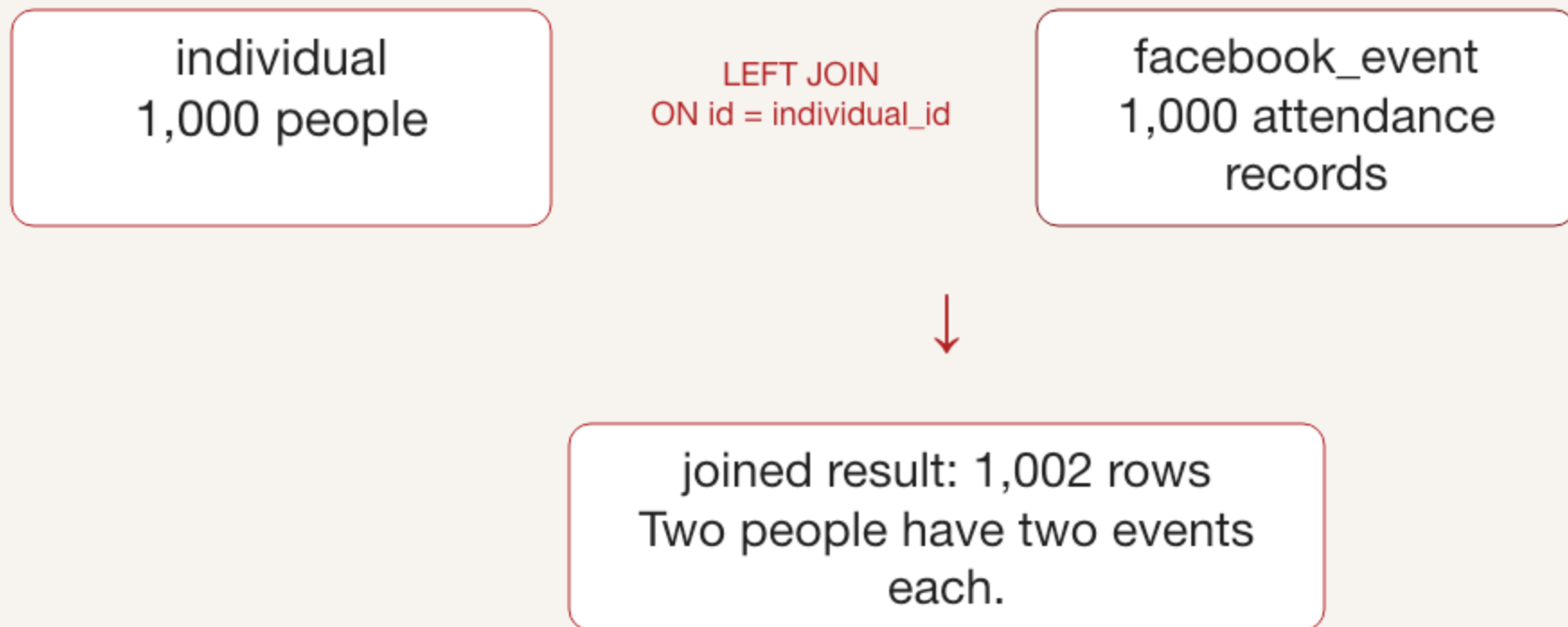
Read left to right: put the table you must keep on the left.

Start with grain, then test the key

- 1 State what one row means in each table.
- 2 Name the primary key and foreign key.
- 3 Predict whether the output should stay the same or grow.
- 4 Inspect a small result with LIMIT.
- 5 Count before you interpret.

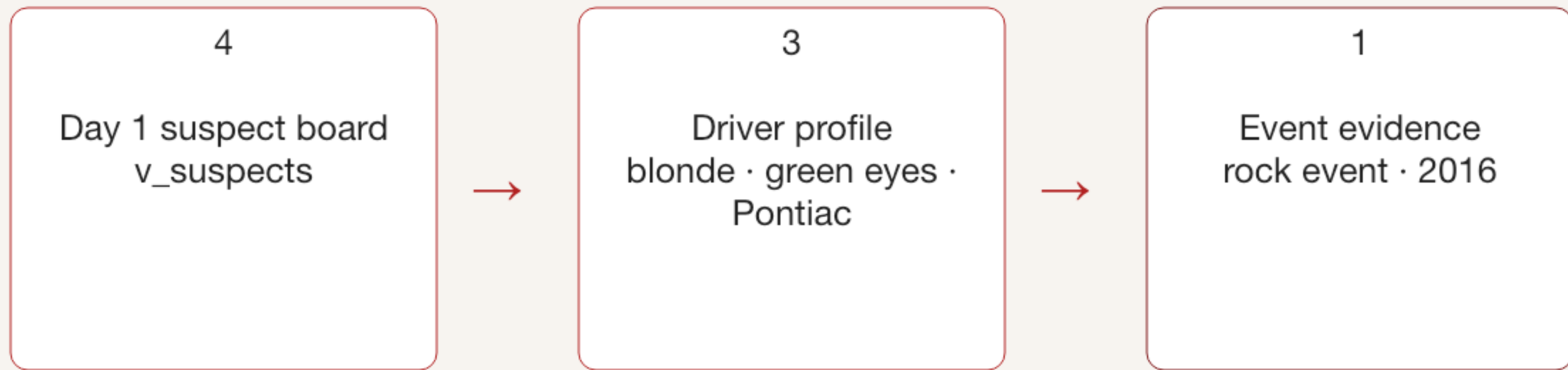
SQL is evidence

A bigger join result can be correct



Do not add DISTINCT to hide a count. Explain the relationship first.

The evidence narrows: $4 \rightarrow 3 \rightarrow 1$



Your conclusion needs an evidence chain, not a query screenshot.

Join validation transfers to BigQuery

SQLite + DBeaver

Small local tables
Fixed counts
Fast feedback

same questions
→

BigQuery Sandbox

Public tables
GoogleSQL
Check the byte estimate

The engine changes. The grain, key, and count checks do not.

Count matched trips, not preserved station rows

```
SELECT
  stations.name,
  COUNT(trips.trip_id) AS trip_count
FROM stations
LEFT JOIN trips
  ON stations.station_id = trips.start_station_id
GROUP BY stations.name;
```

Why not COUNT(*)?

A left join keeps a station even when no trip matches.

COUNT(*) would say 1.
COUNT(trip_id) says 0.

The counted column must represent a real match.

A readable join answers a business question

Which stations launch the most trips?

Join on start_station_id
Group by station name
Count trips

Which stations receive the most trips?

Join on end_station_id
Group by station name
Count trips

Which stations have zero starts?

Start from stations
LEFT JOIN trips
COUNT(trips.trip_id)

Question → grain → key → join type → count check

Before you trust a joined result, ask five questions

- What does one row mean in each input table?
- Which key connects the tables?
- Which table's rows must survive?
- Could the relationship fan out?
- Which count proves the result is plausible?

Next

Use joins as
building blocks for
subqueries and
CTEs.

SQL that runs is only the beginning. SQL you can explain is evidence.